Smart Home Sensing, Time-Series Analytics, and a Digital Twin for Indoor Climate & Energy

MSc Data Science, University of Aberdeen
2025-2026
Nazerke Dalabayeva
Student ID: 52429224

SUPPLEMENTARY MATERIAL

S1. Software, Libraries, and Environment

This project was implemented in Python using widely adopted data science libraries for time-series analysis, machine learning, and data visualization. The primary libraries used are listed below:

- pandas – data manipulation, cleaning, and time-series resampling
- NumPy – numerical computation
- matplotlib and seaborn – data visualisation
- statsmodels – seasonal decomposition and ARIMA modelling
- SciPy – statistical operations such as z-score normalisation
- paho-mqtt – MQTT communication and real-time data streaming
- PostgreSQL – structured storage of telemetry data
- Grafana – real-time dashboard visualisation and alerting
- Xcode / Swift – iOS mobile monitoring interface

The implementation was developed using Python 3.14.0 in a local development environment (VS Code / both macOS & Windows), with data stored in CSV format for reproducibility.

S2. Data Acquisition and Streaming Pipeline

Real-time telemetry data was generated by an ESP32-based smart farmhouse sensing system and transmitted using the Message Queuing Telemetry Transport (MQTT) protocol to a HiveMQ cloud broker. The project focused on receiving, storing, visualising, and analysing the streamed telemetry data.

A Python ingestion workflow was used to receive MQTT messages, decode JSON payloads, normalise telemetry fields, and store the processed records for later analysis. Each message contained environmental sensor readings, event-based signals, and metadata.

The main telemetry fields included:

- timestamp_utc
- device_id
- sequence number
- temperature
- relative humidity
- light intensity
- soil moisture
- water level
- rain indicator
- PIR motion state
- button state
- ultrasonic distance
- RSSI signal strength

This pipeline provided a continuous historical dataset for time-series analysis and system reliability assessment.

S3. Live Monitoring Dashboard

A real-time monitoring dashboard was implemented using Grafana. The dashboard displayed current sensor values and recent trends for temperature, humidity, light intensity, distance, PIR motion status, and RSSI signal strength.

The dashboard supported near-real-time observation of both environmental behaviour and communication health. It formed part of the simplified digital-twin framework by providing a live digital representation of the physical sensing system.

Figure S1. Grafana dashboard used for real-time smart farmhouse telemetry monitoring.

S4. Mobile Application Interface

A mobile monitoring interface was developed in Xcode for iOS. The application connected to the HiveMQ broker using MQTT over WebSocket and displayed selected real-time telemetry values.

The mobile interface included:

- MQTT connection status
- latest temperature and humidity values
- soil moisture status
- RSSI connectivity status
- sequence number
- alert status

Although the mobile application was not part of the core statistical analysis, it demonstrated that the

monitoring framework could be extended to user-facing platforms. This supports the digital-twin concept by allowing remote access to live system status.

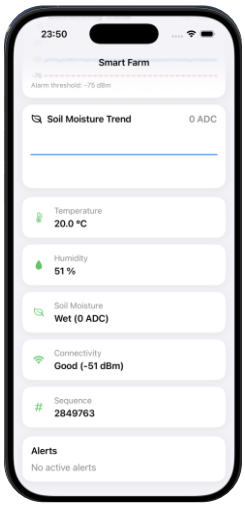

Figure S2. Mobile application interface displaying real-time smart farm telemetry, including temperature, humidity, soil moisture, RSSI connectivity, and system status

S5. Additional Time-Series Analysis

Additional time-series visualizations were generated for all environmental variables using one-minute aggregated data. These include detailed plots and seasonal decompositions that complement the main thesis.

The following figures illustrate the seasonal decomposition results for key environmental variables:

- Figure S3: Seasonal decomposition of temperature
- Figure S4: Seasonal decomposition of humidity
- Figure S5: Seasonal decomposition of RSSI

These figures confirm the presence of daily periodic behaviour and support the findings discussed in the main analysis.

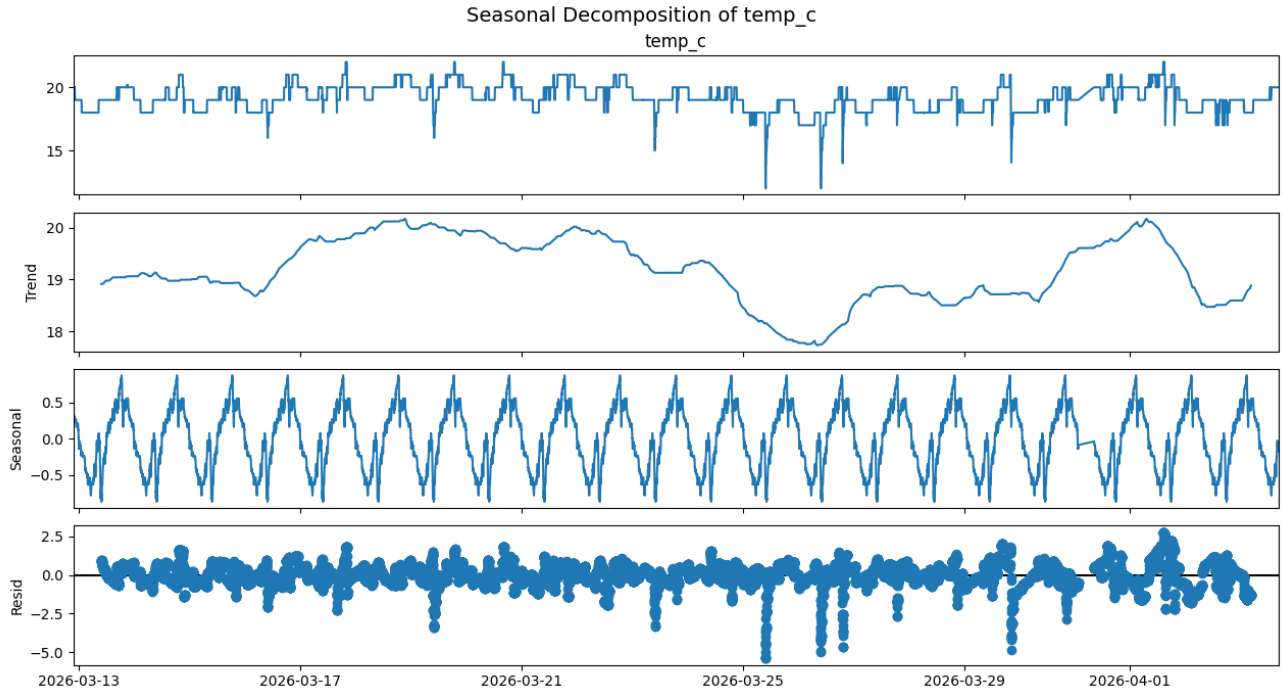

Figure S3. Seasonal decomposition of temperature

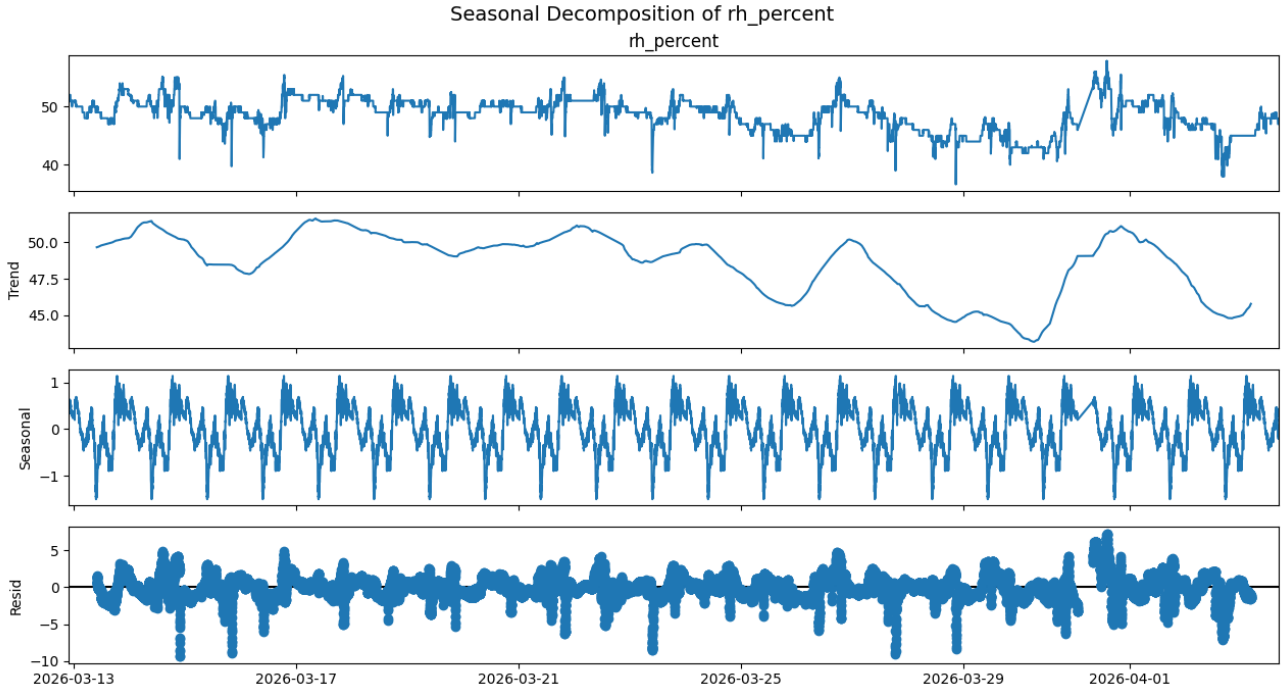

Figure S4. Seasonal decomposition of humidity

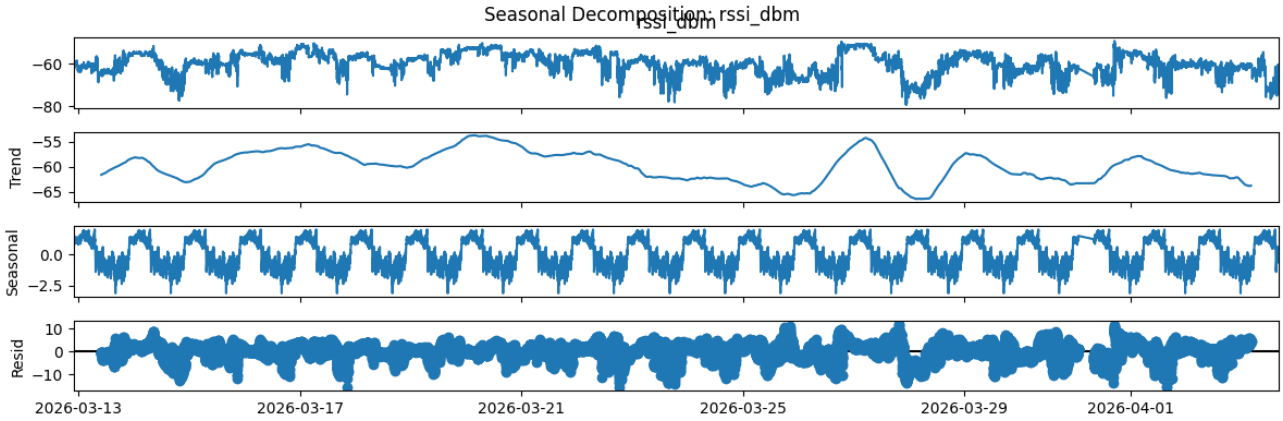

Figure S5. Seasonal decomposition of RSSI

S6. Additional Predictive Monitoring Results

Extended predictive maintenance analysis was performed to evaluate sensor health and system reliability.

Figures included:

- Figure S6: RSSI degradation with rolling mean and threshold detection
- Figure S7: Temperature rolling variance with anomaly thresholds
- Figure S8: Humidity rolling variance indicating sensor instability
- Figure S9: Light sensor behaviour showing flat-line and saturation detection
- Figure S10. Ultrasonic distance sensor predictive monitoring showing temporal variations in measured distance and detection of invalid or unstable readings indicative of potential sensor faults.

These results provide detailed insight into sensor performance and complement the summary presented in the main thesis.

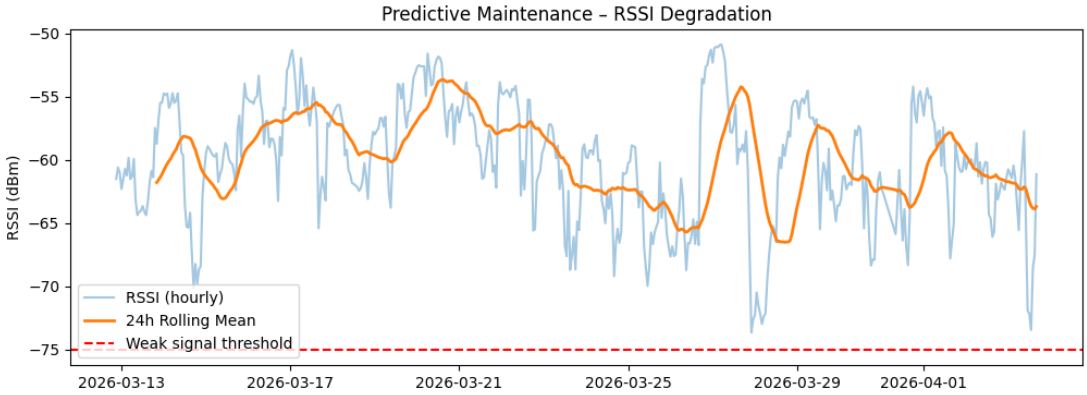

Figure S6: RSSI degradation with rolling mean and threshold detection

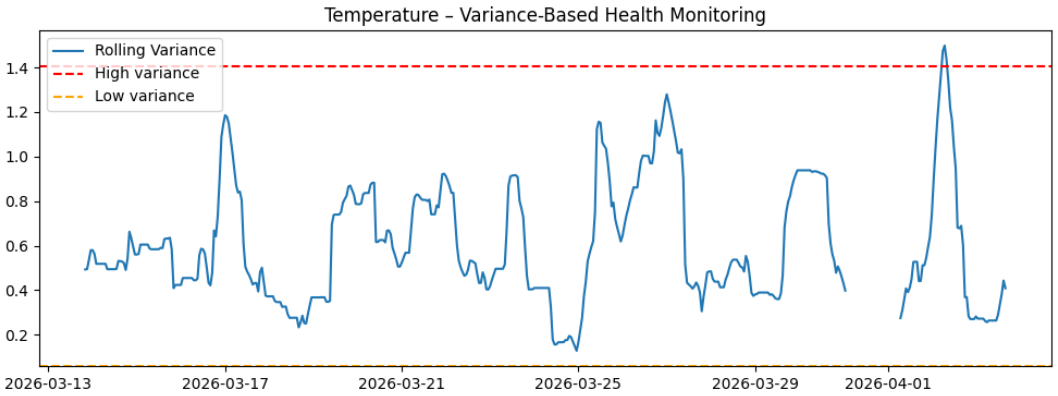

Figure S7: Temperature rolling variance with anomaly thresholds

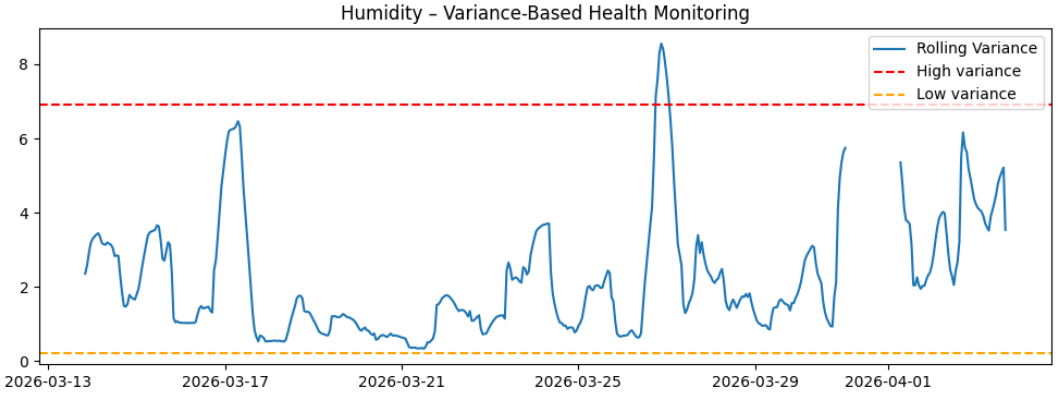

Figure S8: Humidity rolling variance indicating sensor instability

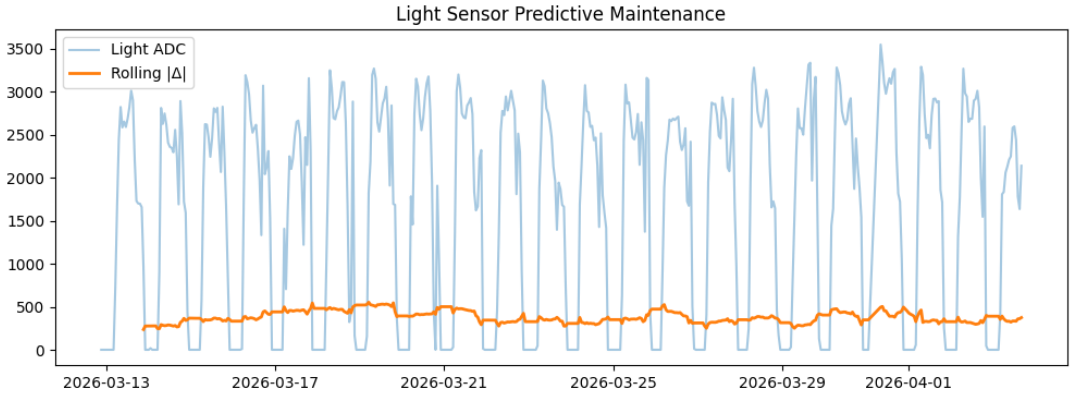

Figure S9: Light sensor behaviour showing flat-line and saturation detection

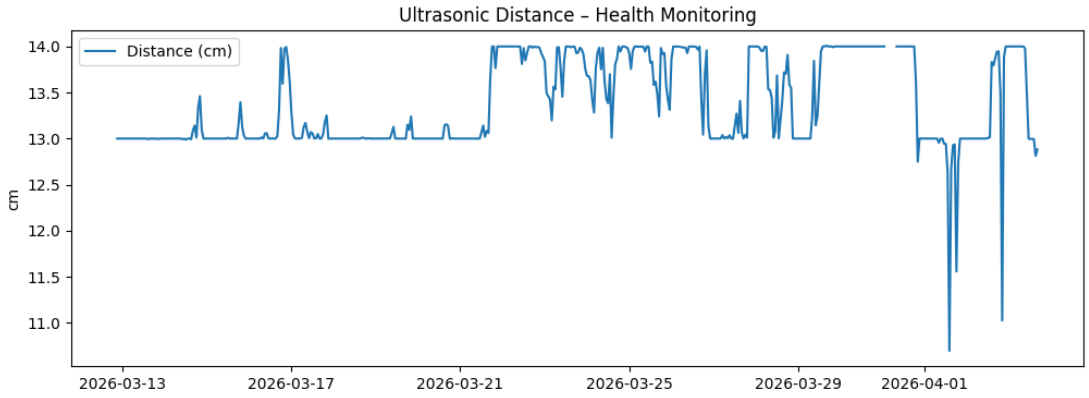

Figure S10. Ultrasonic distance sensor predictive monitoring showing temporal variations in measured distance and detection of invalid or unstable readings indicative of potential sensor faults.

S7. Additional Forecasting Results

Additional forecasting experiments were conducted using decomposition-based ARIMA models for multiple variables.

Figures included:

- **Figure S11**: Decomposition-based ARIMA forecast for RSSI
- **Figure S12**: Decomposition-based ARIMA forecast for humidity
- **Figure S13**: Decomposition-based ARIMA forecast for temperature

  These results further demonstrate the limitations of decomposition-based ARIMA in capturing complex patterns and the improved performance of machine learning models using multivariate lagged features.

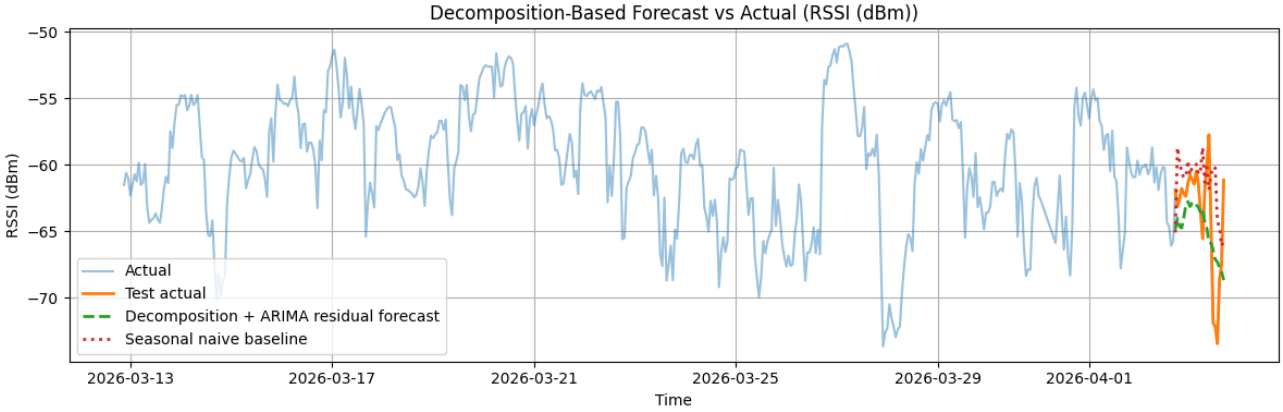

Figure S11: Decomposition-based ARIMA forecast for RSSI

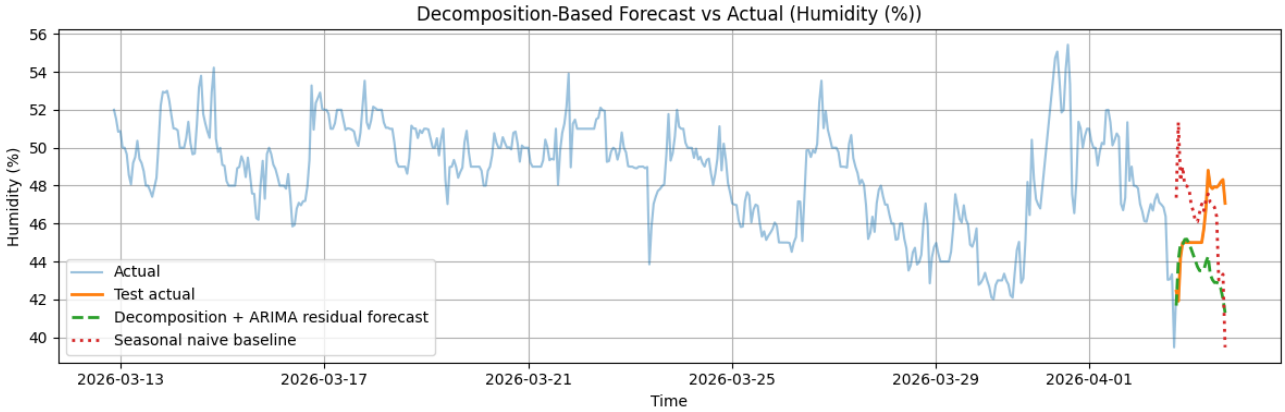

Figure S12: Decomposition-based ARIMA forecast for humidity

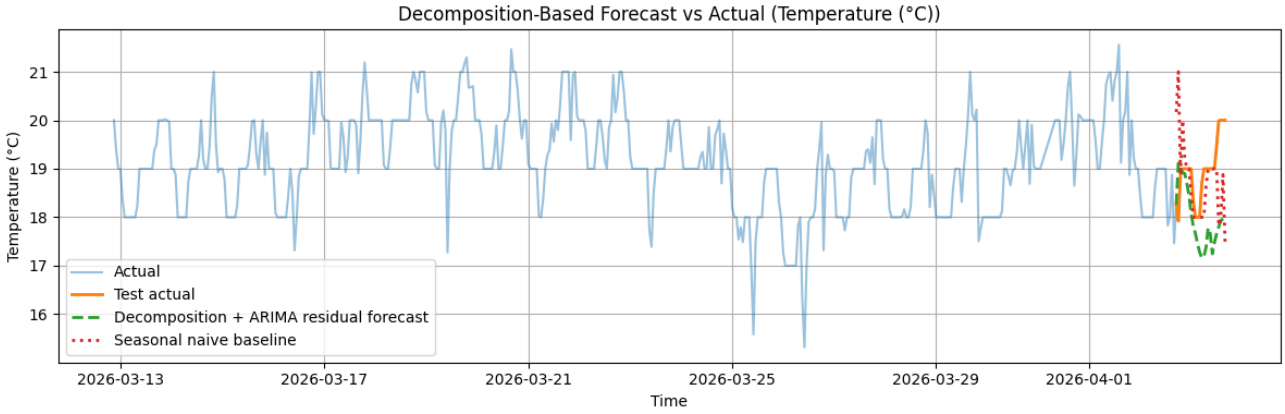

Figure S13: Decomposition-based ARIMA forecast for temperature

S8. Code Availability

The following Python scripts were developed as part of this project:

- live_stream.py – real-time MQTT data ingestion and local storage
- final.py – main descriptive statistics, correlation, lagged cross-correlation, and residual-based anomaly detection pipeline
- button_press_analysis.py – native-resolution button press detection using active-low edge detection
- pir_motion_analysis.py – native-resolution PIR motion detection using active-high edge detection
- predictive_maintenance.py – rule-based sensor health monitoring
- forecasting_models.py – decomposition-based ARIMA forecasting with naive and seasonal-naive baseline comparison

File paths are local to the development environment and should be updated before running the scripts on another machine.

S9. Reproducibility Notes

All analyses were performed using locally stored CSV files generated from the live MQTT telemetry stream. Timestamps were converted to UTC format and used as the time-series index. Data were resampled into one-minute and hourly intervals depending on the analysis. One-minute data were used for detailed time-series behaviour and seasonal decomposition. Hourly data were used for lagged cross-correlation, rule-based sensor monitoring, and forecasting.

Forecasting was evaluated using the final 24 hours as a hold-out test period. Performance was measured using MAE, RMSE, and R^2, and the decomposition-based ARIMA approach was compared with naive and seasonal-naive baselines.